Environnement de développement

Types d’application

Applications de premier plan
- Utilisable quand elle est visible et mis en pause lorsqu’elle ne l’est pas
- Jeux, mashup de cartes géographiques

Applications d’arrière-plan
- Interaction limitée en dehors de sa configuration
- Filtrage d’appels, réponse automatique aux SMS

Applications intermittente
- Interactivé par moment, l’essentiel de sa tâche est en arrière-plan
- Lecteur de musique

Widget
- Composant visuel interactif que les utilisateurs peuvent ajouter à leur écran d’accueil
- Affichage de l’heure, prévisions météo

Les éléments d'une application

- **des activités:** une partie de l'application présentant une vue à l'utilisateur
- **des services :** une activité tâche de fond sans vue associée
- **des fournisseurs de contenus :** permet le partage d'informations au sein ou entre applications
- **des widgets :** une vue accrochée au Bureau d'Android
- **des *Intents :*** *permet d'envoyer un message pour un composant externe* sans le nommer explicitement
- **des récepteurs d'*Intents :*** *permet de déclarer être capable de* répondre à des *Intents*
- **des notifications :** permet de notifier l'utilisateur de la survenue d'événements

Package de l'application

- **src/: les sources Java du projet**
- **libs/: bibliothèques tierces**
- **res/:**
  - res/drawable: ressources images
  - res/layout: description des IHMs en XML
  - res/values: chaines de caractères et dimensions
- **gen/: les ressources auto générées par ADT**
- **assets/: ressources brutes (*raw bytes)***
- **bin/:**
  - bin/classes: les classes compilées en *.class*
  - bin/classes.dex: exécutable pour la JVM Dalvik
  - bin/myapp.zip: les ressources de l'application
  - bin/myapp.apk: application empaquetée avec ses ressource et prête pour le déploiement

Principe R.java

Chaque élément défini dans le répertoire /res impacte le fichier R.java (à ne pas toucher)

Génération automatique de classes internes à la classe R, ainsi que des constantes de type entier sur 32bits

Chemin d'accès aux ressources via R.java

- user-defined : **fr.polytech.tp1.R.color.rose_bonbon**
- system-defined : **android.R.color.darker_gray**
- Exemple : obj.setColor(R.color.rose_bonbon)

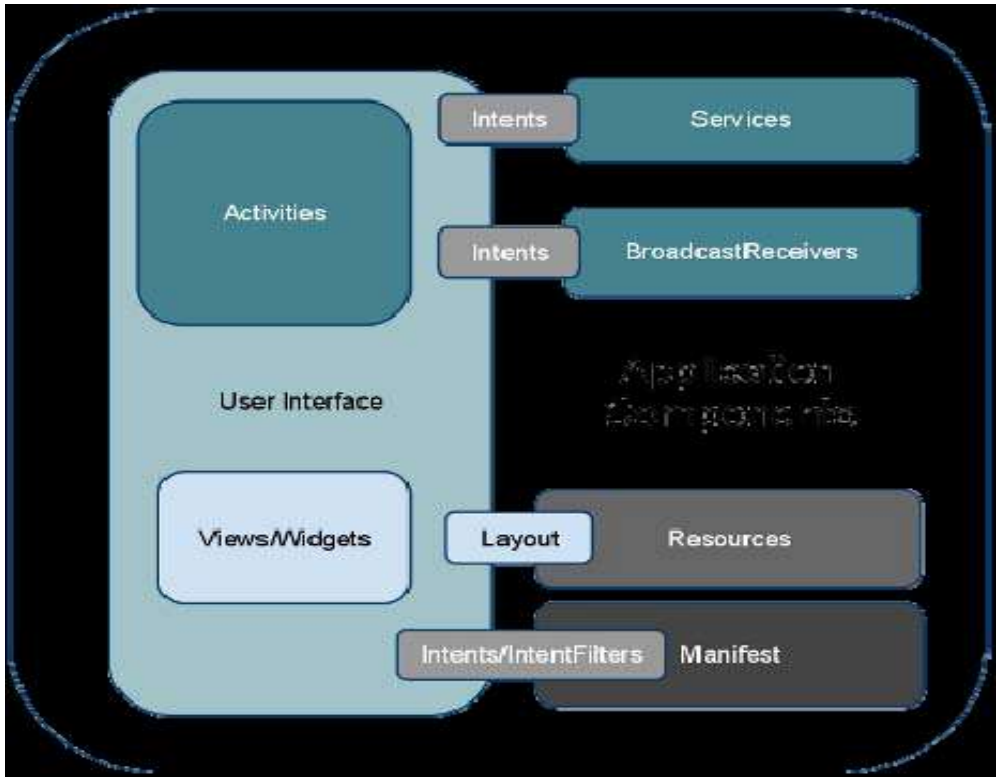
Objet java représentant les ressources

- Instance de la classe : android.content.res.Resources
- Ressources du projet en cours : context.getResources()

AndroidManifest.xml

- Les applications Android sont constitués de composants à couplage.

- Les composants sont liés par un manifeste d'application qui décrit chacun d'eux et comment ils interagissent.



AndroidManifest.xml

- Décrit les composants de l'application
  - Un point d'entrée potentiel dans l'application
  - Une application peut lancer un composant différent (pas de main)
- Quatre types de composants
  - activités (activities)
  - services (services)
  - fournisseurs de contenu (content providers)
  - récepteurs de diffusion (broadcast receivers)
- Sous forme de balises pour chacun des composants
- Décrit les permissions requises par l'application (accès a l'Internet, . . .)
- Décrit les besoins matériels et logiciels de l'application (appareil photo, . . .)

Contrainte de développement

- Les principaux obstacles au développement sont:
  - Les contraintes de test;
  - Le faible nombre d'API disponibles;
  - Les limitations de mémoire;
  - La migration de plate-forme;
  - Des problèmes de performances;
- Tenir compte du matériel :
  - Une puissance processeur faible;
  - Une RAM limitée;
  - Petits écrans avec faibles résolutions;
  - Des coûts élevés de transferts de données;
  - Des connexions réseau moins fiables;
- Efficacité : optimiser votre code afin qu'il soit rapide et réactif.

Les composants graphiques (widjet)

Activity, View, View Group

- **Activity** = écran Android (GUI). Il représente les fenêtres ou les écrans affichés. Une Activity contient des views.

- **View** = composant graphique (Widget). C'est la classe de base, tous les contrôles d'interface sont dérivés de View.

- **View Group** : c'est une extension de la classe View. Il contient plusieurs Views enfants. Son extension permet de :
    1. Créer des contrôles composites interconnectées;
    2. Fournir les gestionnaires de layouts pour disposer les contrôles dans vos Activities;

Views personnalisées

- Android permet de créer des views personnalisés, non disponible sur les composants graphiques de bases.

- L'extension des classes View ou SurfaceView permet de créer des composants views(widgets).

- La classe View fournit un objet Canvas(surface sur laquelle nous dessinons notre vue) ainsi qu'une serie de méthodes de dessin et de classes Paint.

- La classe SurfaceView fournit un objet Surface qui supporte le dessin depuis un thread en arrière-plan et utilise openGL pour la 3D. Elle est excélente pour les contrôles gourmands en graphisme.

Views personnalisées

- Pour créer une vue personnalisée vous pouvez redéfinir les méthodes :
    1. onMeasure : qui calculera la hauteur et la largeur que la vue occupera;
    2. onDraw : qui supportera le Canvas;
- Exemple:

```
@Override
protected void onMeasure(int width,int height){
        [..... Définir la hauteur et la largeur...]
}
@Override
protected void onDraw(Canvas canvas){
        [..... Dessiner votre interface visuelle ...]
}
```

Widgets Android

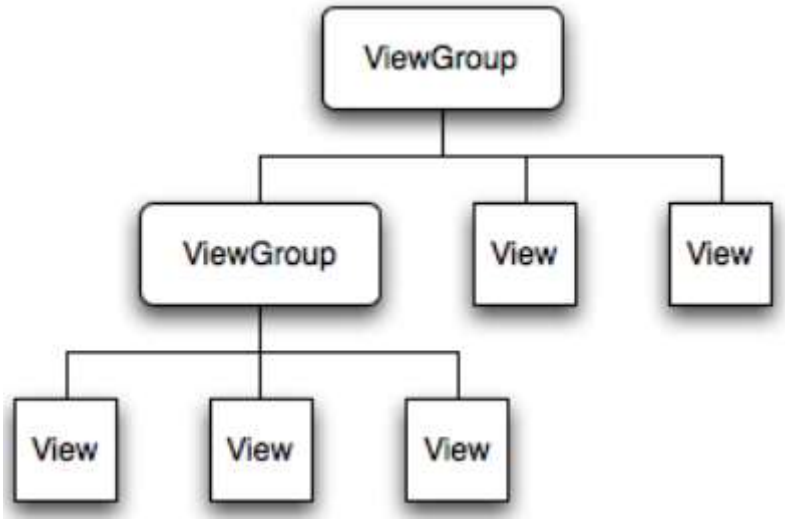
- Android fournit une boîte à outils de Views(widgets) standard qui facilite la création des interfaces;

- Parmis ses contrôles on peut citer :
  1. TextView : pour les labels texte en lecture standard en lecture seule;
  2. EditText : boite de saisie modifiable;
  3. ListView : View Group créant et gérant une liste verticale de Views et les affichant sous forme de lignes dans une liste;
  4. Button : bouton standard;
  5. CheckBox : bouton à deux états représentant une case à cocher;

Les éléments graphiques héritent de la classe View

Regrouper dans une ViewGroup.

Layout (gabarits) : des ViewGroup prédéfinis



Les layouts

La fonction de l’activité

```
public class MainActivity extends ActionBarActivity {
 private TextView texte = null;
   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);
}
```

Layout

Avec des prédispositions des objets graphiques:

- LinearLayout: dispose les éléments de gauche à droite ou du haut vers le bas
- RelativeLayout: les éléments enfants sont placés les uns par rapport aux autres
- TableLayout: disposition matricielle
- FrameLayout: disposition en haut à gauche en empilant les éléments

```
public class Activity2 extends Activity {
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
LinearLayout gabarit = new LinearLayout(this);
gabarit.setGravity(Gravity.CENTER);
// centrer les éléments graphiques
gabarit.setOrientation(LinearLayout.VERTICAL);
// empiler vers le bas !
```

Layouts : Linearlayout, FrameLayout

- Les layouts sont des extensions de la classe **ViewGroup**, utilisés pour placer des contrôles enfants dans une interface utilisateur.

- Le SDK Android inclu quelques layouts simples.

- **Linearlayout** : aligne les views verticalement ou horizontalment. Il contrôle la taille relaive des views.

- **FrameLayout** : c'est le plus simple, il supperpose les views enfants au coin supérieur gauche.

Layouts : RelativeLayout, TableLayout, Gallery

- **RelativeLayout**: c'est le plus flexible des layouts natifs. Il permet dedéfinir les positions chaque view par rapport aux autres et aux limites de l'écran.

- **TableLayout**: permet de placer les views sur une lgrille de lignes et de colonnes.

- **Gallery** : affiche une ligne unique d'objets dans une liste déroulante horizontale.

Layouts : Utilisation

- Les layouts peuvent être implémenter en utilisant du XML ou du code.
- Exemple d'utilisation **LinearLayout** avec XML:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="vertical"
   android:layout_width="fill_parent"
   android:layout_height="fill_parent">
   <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
   android:text="Entrez votre texte ici"
   />
  <EditText
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
   android:text="Le texte s\'affiche là !"
  />
</LinearLayout>
```

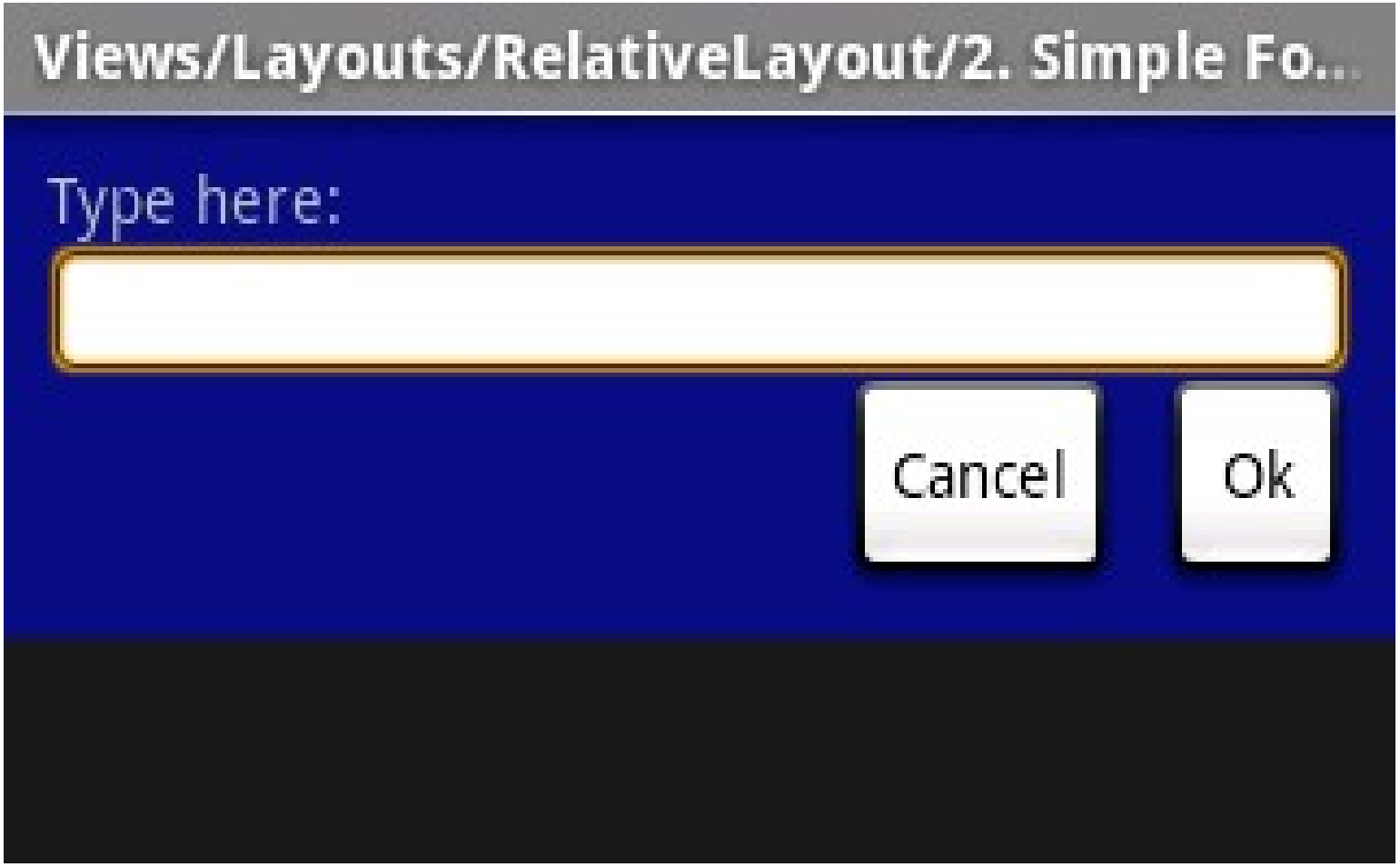
Layouts : Utilisation

- Emple d'utilisation **LinearLayout** avec du code:

```
LinearLayout ll = new LinearLayout(this);
ll.setOrientation(LinearLayout.VERTICAL);
TextView myTextView = new TextView(this);
EditText myEditText = new EditText(this);
myTextView.setText("Entrez votre texte ici");
myEditText.setText("le texte s'affiche là !");
int lHt = LinearLayout.LayoutParam.FILL_PARENT;
int lW = LinearLayout.LayoutParam.WRAP_CONTENT;
ll.addView(myTextView, new LinearLayout.LayoutParams(lH, lW);
ll.addView(myEditTet, new LinearLayout.LayoutParams(lH, lW);
setContentView(ll);
```

- Remarque : La notion de fill_parent a été remplacée par wrap_parent dans la version 2.2 d'Android.

Layouts : Exemple écran



Constraint Layout

- ConstraintLayout est un layout Android introduit pour créer des interfaces **flexibles et performantes**
- Il permet de positionner les vues **les unes par rapport aux autres** ou **par rapport au parent**

Objectifs

- Réduire l'imbrication des layouts
- Faciliter la création d'interfaces complexes
- Améliorer les performances de rendu

Constraint Layout

Contraintes

Chaque vue est positionnée grâce à des **contraintes** (constraints)

Une contrainte relie :

- un côté de la vue (top, bottom, start, end)
- à un autre élément (parent ou autre vue)

Règle clé

Une vue doit avoir **au moins une contrainte horizontale et une verticale**

Outils

Chaînes (*chains*) pour aligner plusieurs vues

Guidelines pour des repères fixes ou proportionnels

Bias pour ajuster la position entre deux contraintes

Constraint Layout

Performances

Layout plus plat → moins de calculs → UI plus rapide

Souplesse

Adaptation facile aux différentes tailles d'écran

Meilleure gestion du responsive design

Productivité

Éditeur visuel puissant dans Android Studio

Moins de code XML comparé aux layouts imbriqués

Recommandation

Layout recommandé par Google pour les interfaces modernes

Contraintes de positionnement

- Définition
  - Les attributs layout_constraint… permettent de **relier un côté d'une vue** à un côté :
    - du **parent**
    - ou d'une **autre vue**

Structure générale

layout_constraint[Côté_vue]_to[Côté_cible]Of

Côtés possibles

- Top, Bottom → axe vertical
- Start, End → axe horizontal

Exemple

- Positionner une vue par rapport au haut du parent :
  - layout_constraintTop_toTopOf="parent"

Contraintes de positionnement

Contraintes verticales

- layout_constraintTop_toTopOf
  - Aligne le **haut** de la vue avec le **haut** de la cible
- layout_constraintBottom_toTopOf
  - Place le **bas** de la vue sur le **haut** d'une autre vue

Contraintes horizontales

- layout_constraintStart_toStartOf
  - Aligne le **début** de la vue avec le **début** de la cible
- layout_constraintEnd_toEndOf
  - Aligne la **fin** de la vue avec la **fin** de la cible

Remarque

- Start / End sont préférés à Left / Right pour la compatibilité RTL

Distance fixe dans les contraintes

Définition

- Une distance fixe correspond à une **marge définie en unités fixes** (dp)
- Elle impose un espacement constant entre deux vues ou entre une vue et le parent

Mise en œuvre

- Utilisation des marges :
    - android:layout_marginTop
    - android:layout_marginBottom
    - android:layout_marginStart
    - android:layout_marginEnd

Exemple :
layout_constraintTop_toTopOf="parent"
android:layout_marginTop="16dp“

Avantage : Simplicité et précision du positionnement

Limite : Moins flexible selon les tailles d’écran

Distance relative dans les contraintes

Définition

- La distance relative dépend de la **position entre deux contraintes**
- Elle s'adapte dynamiquement à la taille de l'écran

Méthodes principales

- **Bias**
  - layout_constraintHorizontal_bias
  - layout_constraintVertical_bias
- **Guidelines**
  - Position fixe ou en pourcentage
- **Chaînes (Chains)**
  - Répartition automatique de l'espace

Exemple (Bias)

```
layout_constraintTop_toTopOf="parent"
layout_constraintBottom_toBottomOf="parent"
layout_constraintVertical_bias="0.3"
```

Avantage : Interfaces plus **responsives et adaptatives**

Exercice :

Créer l'interface suivante :

- Un **TextView** : titre *"Connexion"*
- Deux **EditText** :
- Email
- Mot de passe
- Un **Button** : *"Se connecter"*
- Un **TextView** : *"Mot de passe oublié ?"*

Les contraintes sont :

- Le titre est :
  - Centré horizontalement
  - Placé en haut avec une marge de 32dp
- Les champs :
  - Alignés verticalement
  - Même largeur
  - Espacés de 16dp
- Le bouton :
  - Centré horizontalement
  - Placé sous les champs
- Le texte *"Mot de passe oublié ?"* :
  - Centré horizontalement
  - Aligné en bas de l'écran
  - Utiliser une **contrainte verticale avec bias** ou une **chaîne**

Exercice :

Créer l'interface suivante :

- Un **TextView** : titre *"Connexion"*
- Deux **EditText** :
- Email
- Mot de passe
- Un **Button** : *"Se connecter"*
- Un **TextView** : *"Mot de passe oublié ?"*

Les contraintes sont :

- Le titre est :
  - Centré horizontalement
  - Placé en haut avec une marge de 32dp
- Les champs :
  - Alignés verticalement
  - Même largeur
  - Espacés de 16dp
- Le bouton :
  - Centré horizontalement
  - Placé sous les champs
- Le texte *"Mot de passe oublié ?"* :
  - Centré horizontalement
  - Aligné en bas de l'écran
  - Utiliser une **contrainte verticale avec bias** ou une **chaîne**

Zone de texte :

```
TextView texte = new TextView(this);
texte.setText("Programming creation of interface !");
gabarit.addView(texte);
```

Éditeur de texte :

```
EditText edit = new EditText(this);
edit.setText("Edit me");
gabarit.addView(edit);
```

Les images : doivent être enregistré dans le dossier res

```
ImageView image = new ImageView(this);
image.setImageResource(R.drawable.ensi);
gabarit.addView(image);
```

Button :

- Création :

```
Button b = new Button(this);
edit.setText(“enregistrer");
gabarit.addView(b);
```

- Gestion de l’évènement :

```
Button b = (Button)findViewById(R.id.Button01);
b.setOnClickListener(new OnClickListener() {
@Override
public void onClick(View v) { …  }
});   }
```

Liste de composant :

```
t = new ListView(this);
    ArrayAdapter<String> tableau = new
ArrayAdapter<String>(t.getContext(),
    R.layout.activity_main);
    for (int i=0; i<40; i++) {
    tableau.add("coucou " + i); }
    t.setAdapter(tableau);
```

Gestion des activités

Cycle de vie application Android

- Les applications Android ont un contrôle limité sur leur propre cycle de vie. Leurs composants doivent être à l'écoute des changements d'état de l'application et réagir en conséquence pour une fin intempestive.

- Par défaut, chaque application Android est exécutée dans son propre processus, exécutant une instance distincte de la machine Dalvik.

- Celle-ci se charge de la mémoire et la gestion de processus.

- Les processus sont tués sans avertissements pour libérer des ressources pour d'autres applications.

Priorités des applications

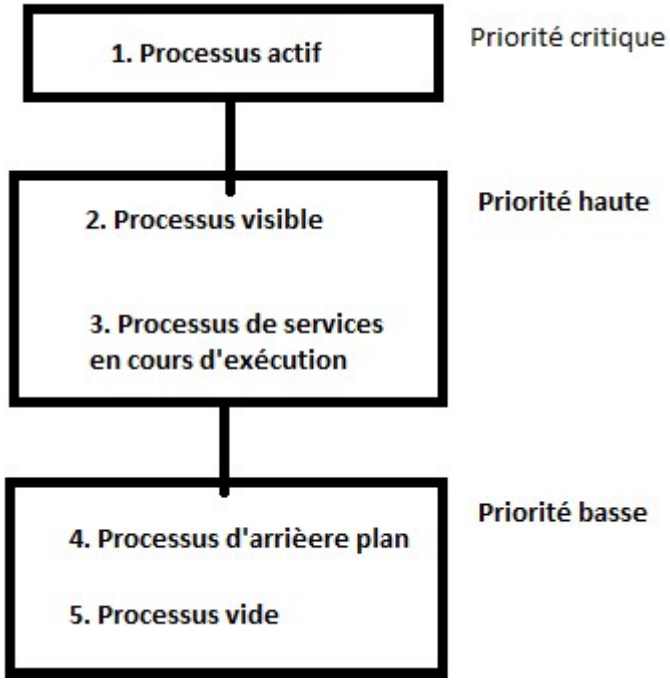
- L'ordre dans lequel les processus sont tués pour libérer les ressources est déterminé par la priorité de leur application.

- La prorité d'une application est celle du composant de plus haute priorité.

- Les priorités peuvent se classer en 3 types:
    1. Priorité critique: le processus est actif;
    2. Priorité haute: le processus est visible ou processus de service en cours d'exécution;
    3. Priorité basse: processus d'arrière plan ou processus vide;

Priorités des applications

- L'arbre des priorités utilisé pour déterminer l'ordre de fin des applications.



Etat des processus

- **Processus actifs** : Sont les processus au premier plan, les composants qui interagissent avec l'utilisateur. Android les gardent à l'état actif en récupérant des ressources.

- **Processus visible** : Son les processus qui hébergent des Activities "visible" mais à l'état inactif. Ils sont tués aux cas extrêms pour permettre aux pressesus actifs decontinuer à s'exécuter.

- **Processus de services en cours d'exécution** : Son les processus hébergeant des services ayant été démarrés. Ils s'exécutent sans interface utilisateur et en continue. Ils sont tués que si des ressources sont necessaires aux processus actifs ou visibles.

Etat des processus

- **Processus d'arrière-plan**: Sont les processus hébergeant des Activities non visible et n'ayant aucun service en cours d'exécution. Ils sont tués pour récupérer des ressources au profit des processus de premier plan.

- **Processus vide** :Pour des raisons de performance, Android garde des applications en mémoire une fois celles-ci terminées. Ce cache réduit le temps de relance des ces applications. Cependant ces processus sont tués si necessaire.

Ressources XML / Java

- Android supporte plusieurs types de ressources (Voir Chapitre des **Ressources**).
- A la compilation d'une application Android, l'outil **AAPT** ou le plugin **ADT** génère la clase R qui contient les Ids des ressources crées.
- Il existe 2 manières d'accéder aux ressources :
    1. Par code : en tuilisant la classe statique R. Exemple **R.string.hello** (où string est le type de ressource et hello le nom de la ressource) ;
    2. Par XML : en utilisant la syntaxe XML qui correspond aussi à l'ID de ressource définie dans laclasse R. Exemple @string/hello (où string est le type de ressource et hello le nom de la ressource);

Classe Application

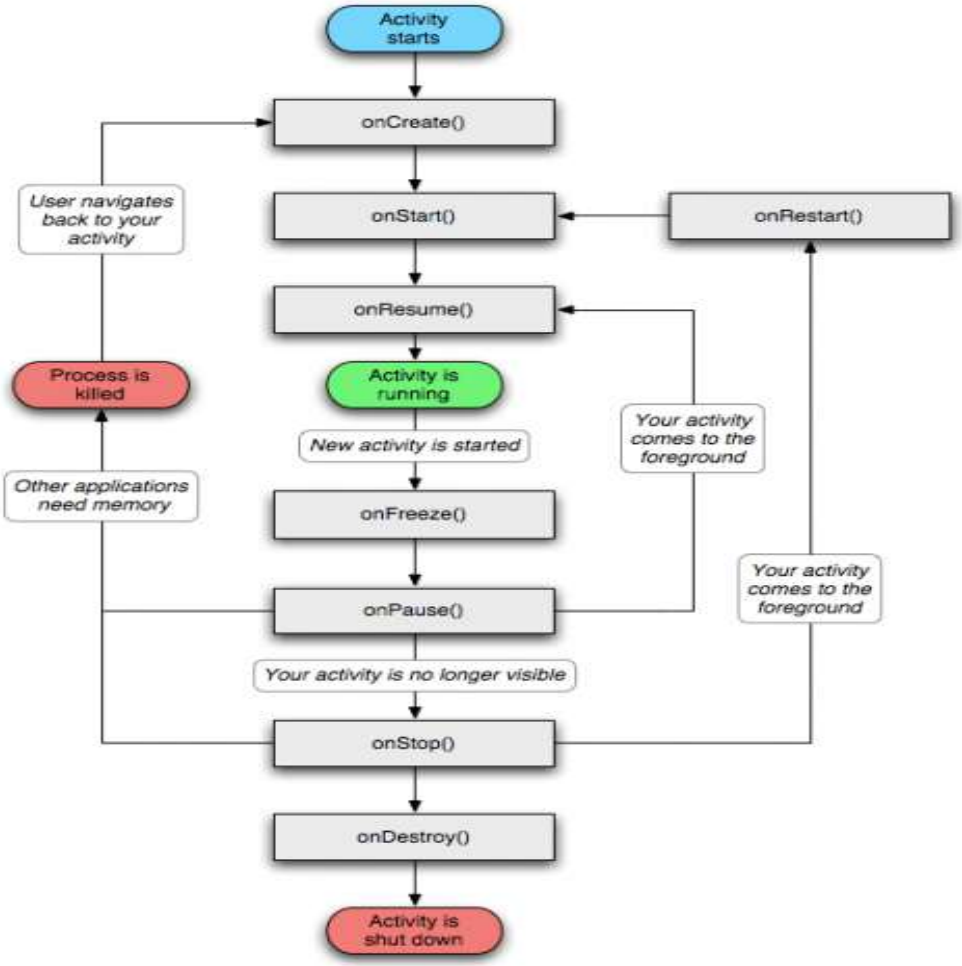
- La classe Application d'Android permet:
  1. Maintenir l'état d'une application;
  2. Transférer des objets entre les composants d'une application;
  3. Gérer et maintenir les ressources utilisées par plusieurs composants d'une application;

- Enregistrez la classe Application dans le manifeste aprés l'avoir implémentée. Ceci donne accès à ses méthodes et variables.

- L'implémentation de la classe est instanciée lorsque le processus de l'application est créé (C'est un singleton).

Activity : le cycle de vie

- Schéma d'un cycle de vie d'une Activity:



Classe Application

- Exemple d'implémentation et d'enregistrement dans le manifeste:

```
Public  class MyApplication extends Application {
   private static Myapplication appSingleton;
   //renvoie l'instance de l'application
   public static MyApplication getInstance() {
        return appSingleton;
   }
   @override
   public final void onCreate() {
        super.onCreate();
        appSingleton=this;
   }
}
```

Classe Application

- Exemple d'implémentation et d'enregistrement dans le manifeste: (suite)

```
<application
   android:icon="@drawable/icon"
   android:name="MyApplication">
   [...Noeuds du manifeste....]
</application>
```

- Ainsi vous pouvez créer de nouvelles variables d'état et de ressources globales pour y accéder depuis les composants de l'application.

Exemple : MyObject value = MyApplication.getInstance().getGlobalStateValue();
MyApplication.getInstance().setGlobalStateValue(myObectValue);

Classe Application

- La classe application fournit également des gestionnaires d'évenements pour le démarrage et la fin d'une application, de la méoire disponible et de changement de configuration:
    1. onCreate : pour le démarrage de l'application;
    2. onTerminate : pour la fin de l'application;
    3. onLowMemory : fournit aux applications pour libérer de la mémoire lorsque le système manque de resource;
    4. onConfigurationChanged : qui permet de gérer les changement de configuration au niveau de l'application;

Activity : la classe

- La classe Activity permet de créer des écrans en utilisant des Views pour interagir avec l'utilisateur.

- Chauqe Activity représente un écran (Form).

- Exemple :

```
public class MyActivity extends Activity {
   //Appelée lorsque l'activité est crée pour la 1ere fois
   @override
   public void onCreate(Bundle saveInstanceState){
        super.onCreate(saveInstanceState);
   }
}
```

Activity : le cycle de vie

- Une bonne compréhension du cycle de vie de l'Activity permet de garantir que votre application satistfait vos utilisateur et gère correctement ses ressources.

- L'état de chaque Activity est déterminé par sa position dans la pile des Activities:
    1. Active : Quand l'Activity est au sommet de la pile;
    2. En pause : Si l'Activity est visible sans avoir le focus;
    3. Stoppée : Lorsque l'Activity n'est plus visible;
    4. Inactive : Quand elle est tuée ou qu'elle n'est pas démarrée;

Ressources drawable

- Android contient plusieurs types de ressources Drawable.

- Les ressources sont sont stockées dans le dossier res/drawable.

- Les ressouces drawable peuvent être définies et manipulées par du XML ou du code.

Ressources drawable

- Exemple d'utilisation avec une ressource image.

```
<ImageView
   android:layout_height="wrap_content"
   android:layout_width="wrap_content"
   android:src="@drawable/myimage" />

Resources res = getResources();
Drawable drawable = res.getDrawable(R.drawable.myimage);
```

- Où le referencement :
  - En Java : R.drawable.myimage
  - En XML : @drawable/myimage

Résolution

- 2009-2010 constitue une explosion du nombre d'appareils Android(large gamme de résolutions d'écran).
- Les interfaces développées doivent tenir compte de cette large gamme de résolution.
- Les applications Android doivent pouvoir s'exécuter sur une variété de matériels indépendamment.
- Le framework Android fournit des techniques qui permettent d'optimiser l'interface utilisateur pour fonctionner dans différentes tailles d'écran.

Résolution

- Qualificateurs de ressources : Structurer votre projet en créant des répertoires pour stocker vos ressources externes pour différentes configurations matérielles.

- Exemple de qualificateurs :
  1. res/layout-small-long/ : Layout pour de petits et longs écrans;
  2. res/layout-large/ : Layout pour de grands écrans;
  3. res/drawable-hdpi/ : Drawables pour des écrans à haute densité;

Résolution

- Possibilité d'optimiser vos interfaces en utilisant le fichier manifest.

- Exemple :

```
<supports-screens
   android:smallScreens="false"
    android:normalScreens="true"
    android:largeScreens="true"
    android:anyDensity="true"
/>
```

- L'attribut anyDensity contrôle comment l'application sera mise à l'echelle quand elle s'affiche sur des appareils de densités diverses.